

计算机组成原理

计算机系统概述
数据的表示和运算

考试大纲

一、计算机系统概述

(一) 计算机系统层次结构

1. 计算机系统的基本组成
2. 计算机硬件的基本组成
3. 计算机软件和硬件的关系
4. 计算机系统的工作原理

“存储程序”工作方式，高级语言程序与机器语言程序之间的转换，程序和指令的执行过程

(二) 计算机性能指标

吞吐量、响应时间； CPU 时钟周期、主频、 CPI 、 CPU 执行时间； MIPS 、 MFLOPS

GFLOPS 、 TFLOPS、 PFLOPS、 EFLOPS、 ZFLOPS。

二、数据的表示和运算

(一) 数制与编码

1. 进位计数制及其数据之间的相互转换
2. 定点数的编码表示

(二) 运算方法和运算电路

考试大纲

1. 基本运算部件

加法器，算术逻辑部件（ALU）

2. 加/减运算

补码加/减运算器，标志位的生成。

3. 乘/除运算

乘/除法运算的基本原理，乘法电路和除法电路的基本结构。

（三）整数的表示和运算

1. 无符号整数的表示和运算

2. 带符号整数的表示和运算

（四）浮点数的表示和运算

1. 浮点数的表示

IEEE 754 标准

2. 浮点数的加/减运算

1.1.1 计算机系统的基本组成

🔧 硬件系统 (Hardware)

计算机系统的物理实体，是软件运行的物质基础：

- ✓ **运算器**：执行算术逻辑运算 (ALU)。
- ✓ **控制器**：指挥全机指挥中心，译码指令。
- ✓ **存储器**：存放程序和数据 (主存/外存)。
- ✓ **输入/输出设备**：人机交互与信息采集。

💻 软件系统 (Software)

指挥硬件工作的程序、数据及相关文档：

- » **系统软件**：管理计算机资源 (操作系统、编译器、数据库管理系统)。
- » **应用软件**：解决特定应用领域问题 (浏览器、办公软件、算法程序)。
- ⚠ **陷阱**：数据库管理系统属于系统软件，而非应用软件！

1.1.2 冯·诺依曼计算机的核心特点



存储程序工作方式

程序和数据以同等地位预先存放在存储器中，按地址寻址，CPU自动连续执行指令。

二进制形式表示

指令和数据均采用二进制代码表示。指令由操作码 (OP) 和地址码 (Ad) 组成。



五大基本部件

以**运算器为中心**（现代计算机已演变为以**存储器为中心**），由五大硬件部件构成。

1.1.2 现代计算机硬件组成与组件关系

硬件核心组件	内部关键构件 / 寄存器	功能描述
CPU (中央处理器)	ALU, CU, PC, IR, MAR, MDR, ACC	逻辑运算与整机控制，PC存放下一条指令地址。
主存储器 (Main Memory)	存储体 (存储单元+存储元件), MAR, MDR	按地址随机存取，MAR存放地址，MDR存放数据。
I/O 设备 (Input/Output)	I/O 接口 (控制器), 缓冲区	实现主机与外部世界的数据交换与电信号转换。

关键区分： MAR (Memory Address Register) 和 MDR (Memory Data Register) 逻辑上属于存储器，但在现代集成电路中通常集成在 CPU 芯片内部。

1.1.3 计算机软硬件的关系与逻辑等效性

 **逻辑等效性 (Logical Equivalence):** 任何由软件完成的操作, 理论上都可以由硬件直接实现; 反之亦然。

 **权衡选择 (Trade-off):**

- 硬件实现: 速度快, 成本高, 灵活性差。
- 软件实现: 速度较慢, 成本低, 极易升级扩充。

 **软硬件界面:** 指令集架构 (ISA - Instruction Set Architecture) 是硬件与软件之间的核心界面。

计算机系统的多级层次结构

M4: 高级语言机器 - 执行高级语言 (C/C++/Java)

M3: 汇编语言机器 - 执行汇编语言程序

M2: 操作系统机器 - 扩充机械指令 (软硬件交界)

M1: 传统机器语言 - 执行机器指令

M0: 微程序机器 - 微指令控制硬件电路

1.1.4 计算机的工作原理与语言转换

1

高级语言源程序

C/C++ 代码
(人易读, CPU不认识)

2

编译 / 汇编

编译器转换为目标代码
汇编器转为机器码

3

链接器装载

合并库文件与目标文件
生成可执行二进制文件

4

CPU 执行指令

取指 (Fetch)
译码 (Decode)
执行 (Execute)



1.2.1 计算机主要性能指标：时间与频率

🕒 时钟周期与主频

CPU 时钟周期 (T_{clk}): CPU工作的最基本时间单位 (秒/周期)。

CPU 主频 (f): 时钟周期的倒数, 单位 Hz / MHz / GHz。

$$f = \frac{1}{T_{clk}}$$

📊 CPI 与 CPU 执行时间

CPI (Cycles Per Instruction): 执行一条指令所需的平均时钟周期数。

CPU 时间 (TCPU): 执行程序所消耗的 CPU 时间。

$$T_{CPU} = IC \times CPI \times T_{clk} = \frac{IC \times CPI}{f}$$

1.2.2 性能指标：MIPS 与 FLOPS 换算表

指标缩写	全称与含义	换算公式 / 逻辑关系
MIPS	Million Instructions Per Second (每秒百万条指令)	$MIPS = \frac{\text{指令数}}{(\text{执行时间} \times 10^6)} = \frac{f}{(CPI \times 10^6)}$
MFLOPS	Mega FLOPS (10 ⁶ 次浮点运算/秒)	基础浮点性能单位
GFLOPS / TFLOPS	Giga (10 ⁹) / Tera (10 ¹²) FLOPS	主流显卡与CPU计算性能衡量
PFLOPS / EFLOPS / ZFLOPS	Peta (10 ¹⁵) / Exa (10 ¹⁸) / Zetta (10 ²¹)	超级计算机 (HPC) 与 AI 算力集群性能衡量

2.1.1 进位计数制及其相互转换



二进制/八进制/十六进制

- $2^3 = 8$: 1 位八进制对应 3 位二进制。
- $2^4 = 16$: 1 位十六进制对应 4 位二进制。



十进制转 r 进制

- **整数部分**: 除 r 取余法, 先得低位, 后得高位 (倒序排列)。
- **小数部分**: 乘 r 取整法, 先得高位, 后得低位 (正序排列)。



真值与机器数

- **真值**: 带有 '+'、'-' 符号的实际数值。
- **机器数**: 符号数值化, 用 '0' 表示正, '1' 表示负。

2.1.1 BCD 码与字符/汉字编码

编码类型	编码规则与特征	考研核心注意点
8421 BCD 码	用 4 位二进制表示 1 位十进制 (0000~1001)	相加结果 > 9 (1001) 时, 必须加 6 (0110) 进行修正!
余3码 / 2421码	余3码=8421码+0011; 2421码为自补码	2421码中 0101~1010 为非法码。
ASCII 码	7 位二进制编码, 表示 128 个字符	数字 '0'=48, 'A'=65, 'a'=97 ('a' 比 'A' 大 32)。
汉字编码	输入码 → 国标码 (GB2312) → 机内码 → 字形码	机内码: 国标码的最高位置 '1' (避免与 ASCII 冲突)。

2.1.2 定点数的编码表示：原码与反码

原码 (Sign-Magnitude)

定义：最高位为符号位 (0正1负)，其余各位表示数值绝对值。

零的表示：有 +0 (00000) 和 -0 (10000) 两种表示。

表数范围 (8位定点整数)： $-(2^7-1) \sim +(2^7-1)$ ，即 $-127 \sim +127$ 。

反码 (1's Complement)

定义：正数反码同原码；负数符号位为 1，数值位按位取反。

零的表示：同样存在 +0 (00000) 和 -0 (11111) 两种。

应用场景：主要作为求取补码的中间过程，极少直接用于计算。

2.1.2 定点数的编码表示：补码与移码

补码 (2's Complement)

定义：正数补码同原码；负数补码 = 反码 + 1。

零的唯一性： $[+0]_{\text{补}} = [-0]_{\text{补}} = 00000000$ 。

多出的表数：10000000 表示 -128 (8位)。

表数范围 (8位定点整数)： $-2^7 \sim +(2^7-1)$ ，即 $-128 \sim +127$ 。

移码 (Excess Code)

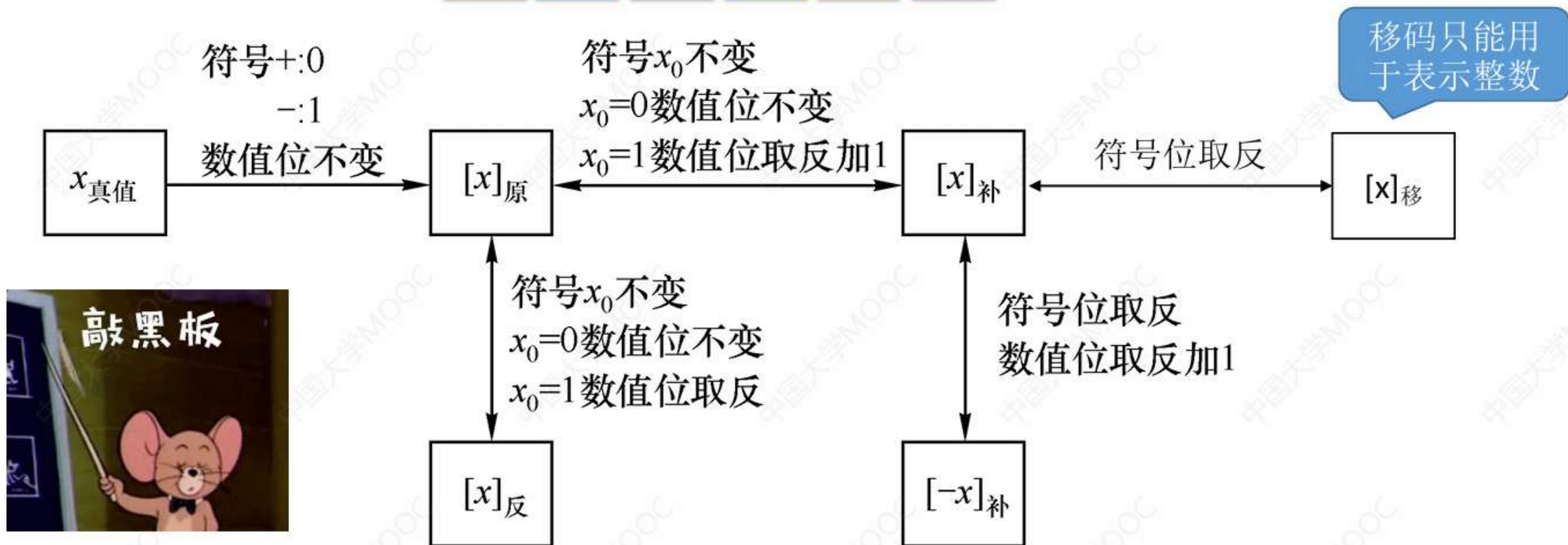
定义： $[X]_{\text{移}} = 2^n + X$ (加上偏置常数 2^n)。

与补码关系：仅符号位相反，数值位完全相同！

应用场景：专用于表示浮点数的阶码，便于比较大小。

2.1.2 四种定点数编码方式汇总对比

码制	正数表示	负数表示	零的表示	8位定点整数表示范围
原码	0 + 绝对值	1 + 绝对值	不唯一 (+0: 00..0, -0: 10..0)	-127 ~ +127
反码	0 + 绝对值	1 + 绝对值取反	不唯一 (+0: 00..0, -0: 11..1)	-127 ~ +127
补码	0 + 绝对值	1 + 反码+1	唯一 (00000000)	-128 ~ +127
移码	1 + 绝对值	0 + 补码数值位	唯一 (10000000)	-128 ~ +127



原码和反码的真值0有两种表示；补码和移码的真值0只有一种表示。若机器字长为 $n+1$ 位，则：

原码和反码

补码

移码

——整数表示范围 $-(2^n-1) \leq x \leq 2^n-1$ ；小数表示范围 $-(1-2^{-n}) \leq x \leq 1-2^{-n}$

——整数表示范围 $-2^n \leq x \leq 2^n-1$ ；小数表示范围 $-1 \leq x \leq 1-2^{-n}$

——整数表示范围 $-2^n \leq x \leq 2^n-1$ ；移码全0真值最小，移码全1真值最大

2.2.1 基本运算部件：加法器与进位逻辑

一位全加器 (FA - Full Adder)

接收输入 A_i , B_i 和低位进位 C_{i-1} , 输出本位和 S_i 及高位进位 C_i :

$$S_i = A_i \oplus B_i \oplus C_{i-1}$$

$$C_i = A_i B_i + (A_i \oplus B_i) C_{i-1}$$

进位传递方式对比

» 串行进位 (RCA - Ripple Carry Adder):

高位进位依赖低位进位产生, 延迟随位数线性增加 ($O(n)$), 速度慢。

⚡ 并行进位 (CLA - Carry Lookahead Adder):

通过进位生成项 $G_i = A_i B_i$ 和传递项 $P_i = A_i \oplus B_i$, 展开逻辑电路直接并行生成各位进位, 延迟极低。

2.2.1 算术逻辑部件 (ALU) 功能结构

🔧 ALU 核心功能

- ✓ **算术运算**: 加、减、带进位加/减、增1、减1等。
- ✓ **逻辑运算**: 与 (AND)、或 (OR)、非 (NOT)、异或 (XOR)。
- ✓ **移位操作**: 逻辑左/右移、算术左/右移、循环移位。

🔌 ALU 控制信号

ALU 是组合逻辑电路，无内部存储状态：

- **输入**: 数据操作数 A, B + 控制信号 Select + 低位进位 Cin。
- **输出**: 运算结果 Result + 条件标志位 (OF, SF, ZF, CF)。

2.2.2 补码加/减运算器实现原理

 **统一加减法定理：** $[A - B]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}}$ 。

 **硬件控制逻辑 (Sub 信号控制)：**

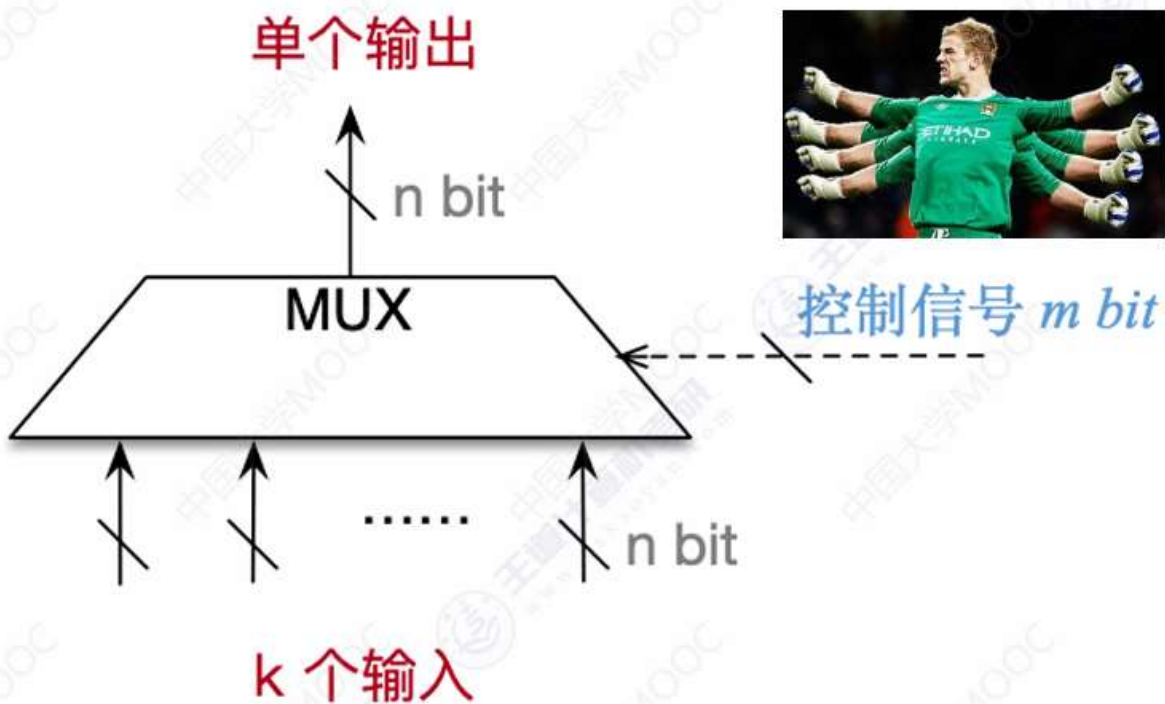
当执行加法 (Sub=0) 时，输入 B 直接进入加法器，末位进位 $C_{in} = 0$ 。

当执行减法 (Sub=1) 时，输入 B 经过**异或门按位取反** ($B \oplus 1 = \bar{B}$)，且末位进位 $C_{in} = 1$ (实现 +1)。

 **设计精髓：** 利用一套加法器硬件电路同时实现定点数补码的加法和减法运算！

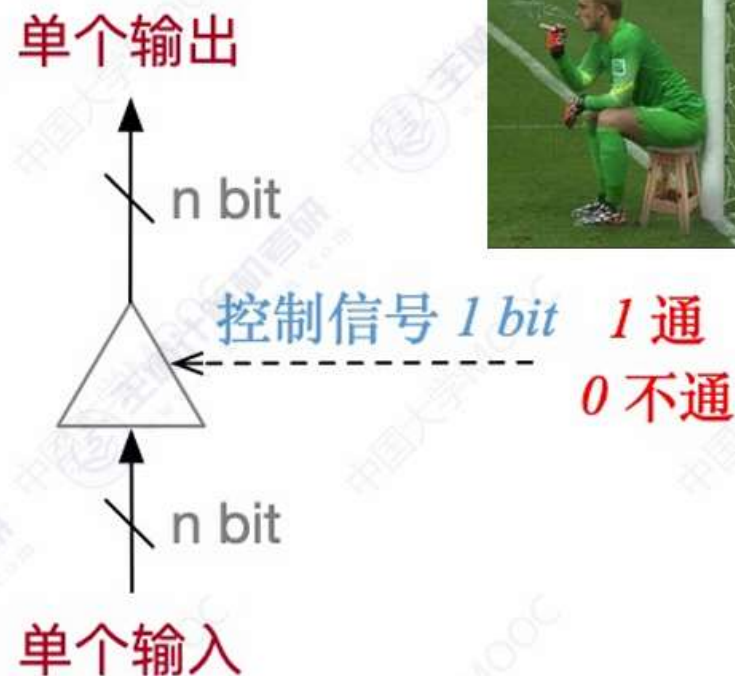
2.2.2 标志位 (OF, SF, ZF, CF) 的硬件生成

标志位	标志含义	硬件生成判定逻辑逻辑式
OF (Overflow)	有符号数 溢出标志	$OF = C_{top} \oplus C_{top-1}$ (最高数值位进位 $\oplus$ 符号位进位)
SF (Sign)	符号标志 (结果正负)	$SF = F_{最高位}$ (直接等于结果最高位)
ZF (Zero)	零标志 (结果是否全0)	$ZF = 1$ (当且仅当运算结果所有位均为 0)
CF (Carry)	无符号数 溢出/借位标志	$CF = C_{out} \oplus Sub$ (加法时 $C_{out}=1$ 溢出, 减法时 $C_{out}=0$ 借位)



若多路选择器有 k 个输入，则
控制信号的位数 $m \geq \lceil \log_2 k \rceil \text{ bit}$

注：有的多路选择器可能会预留一个控制信号，用于拦截所有输入



三态门的控制信号的仅需 1bit

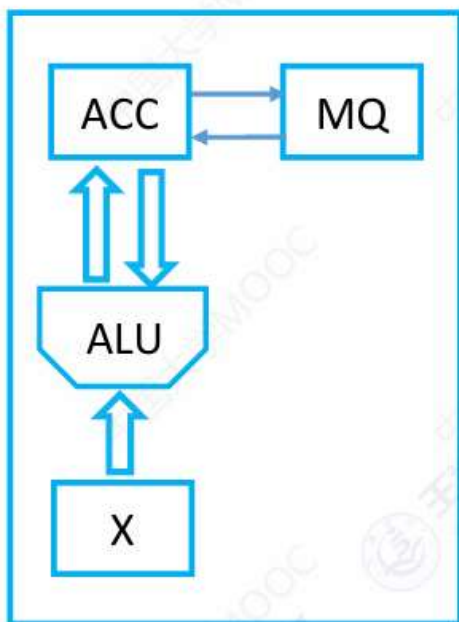


穿越：运算器的基本组成



缓缓地回忆过去

运算器



运算器：用于实现算术运算（如：加减乘除）、逻辑运算（如：与或非）

ACC：累加器，用于存放操作数，或运算结果。

MQ：乘商寄存器，在乘、除运算时，用于存放操作数或运算结果。

X：通用的操作数寄存器，用于存放操作数

ALU：算术逻辑单元，通过内部复杂的电路实现算数运算、逻辑运算

Accumulator
Multiple-Quotient Register
Arithmetic and Logic Unit

	加	减	乘	除
ACC	被加数、和	被减数、差	乘积高位	被除数、余数
MQ			乘数、乘积低位	商
X	加数	减数	被乘数	除数

2.2.3 乘法运算原理与 Booth 算法

无符号数阵列乘法器

- 类似于人工笔算乘法：对乘数每一位进行“与”操作生成部分积。
- 部分积累加并右移，通过 $n \times n$ 的加法器阵列并行计算。

补码 Booth 乘法算法

- 引入辅助位 Y_{n+1} ，根据 (Y_i, Y_{i+1}) 的状态决定操作：

00 / 11: 仅右移 1 位；

01: 部分积 + $[X]_{\text{补}}$ ，再右移 1 位；

10: 部分积 + $[-X]_{\text{补}}$ ，再右移 1 位。



原码一位乘法

设机器字长为 $n+1=5$ 位（含1位符号位）， $[x]_{\text{原}} = 1.1101$ ， $[y]_{\text{原}} = 0.1011$ ，采用原码一位乘法求 $x \cdot y$

符号位	数值位
-----	-----

符号单独处理：符号位 = $x_s \oplus y_s$

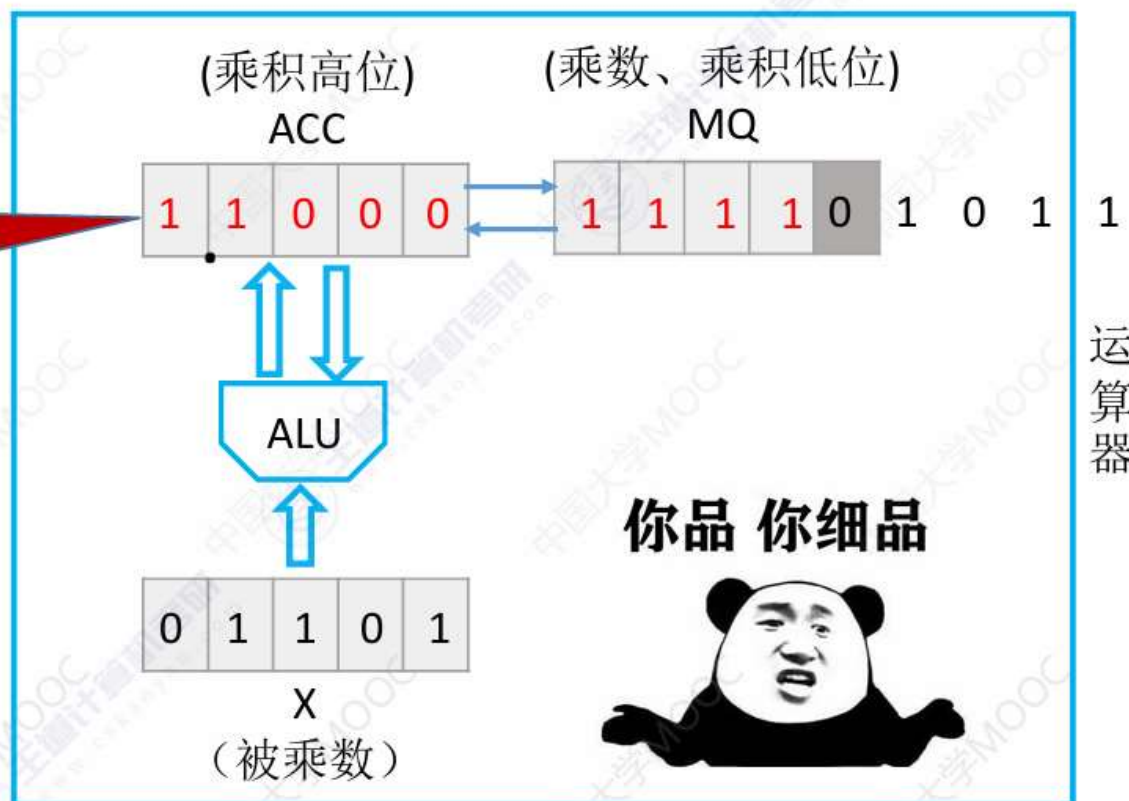
数值位取绝对值进行乘法计算

实现方法：先加法再移位，重复n次

```
  0.1101
× 0.1011
-----
  01101
 01101
 00000
 01101
-----
0.10001111
```

当前位=1，则ACC加上被乘数
当前位=0，则ACC加上 0

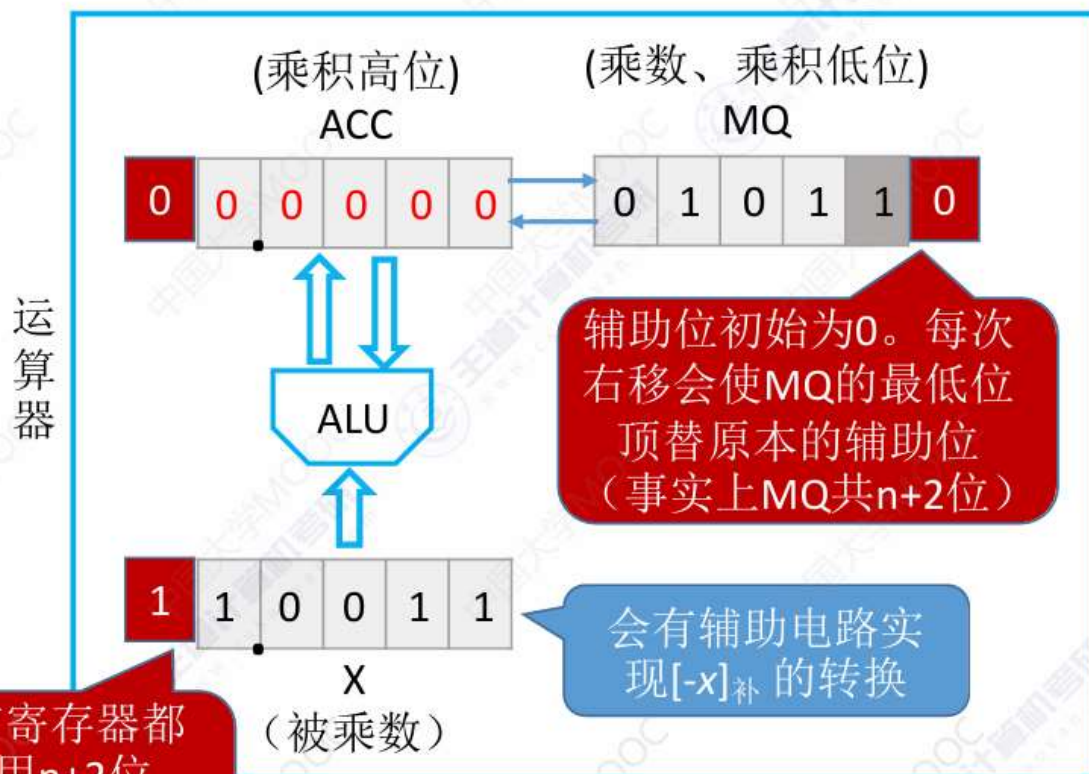
修改符号
位 $x_s \oplus y_s = 1$



补码一位乘法

设机器字长为5位（含1位符号位， $n=4$ ）， $x = -0.1101$ ， $y = +0.1011$ ，采用Booth算法求 $x \cdot y$

$[x]_{\text{补}} = 1.0011$ ， $[-x]_{\text{补}} = 0.1101$ ， $[y]_{\text{补}} = 0.1011$



所有寄存器都统一用 $n+2$ 位，因此采用双符号位补码运算

补码一位乘法：

进行 n 轮加法、移位，最后再多来一次加法

每次加法可能 $+0$ 、 $+[x]_{\text{补}}$ 、 $[-x]_{\text{补}}$

每次移位是“补码的算数右移”

符号位参与运算

根据当前MQ中的最低位、辅助位 来确定加什么

辅助位 - MQ中最低位 = 1时， $(ACC) + [x]_{\text{补}}$
辅助位 - MQ中最低位 = 0时， $(ACC) + 0$
辅助位 - MQ中最低位 = -1时， $(ACC) + [-x]_{\text{补}}$

2.2.3 除法运算：恢复余数与加减交替法

🔄恢复余数法 (Restoring)

1. 默认上商 1，做减法 (余数 - 除数)。
2. 若余数为负，说明不够减，上商改为 0，并加上除数恢复余数。
3. 步数不固定，电路控制极其复杂。

⚡加减交替法 (Non-restoring)

1. 若上一步余数为正，上商 1，下位左移并减除数。
2. 若上一步余数为负，上商 0，下位左移并加除数。
3. 步数固定，运算速度快，现代计算机广泛采用。

原码除法：恢复余数法

设机器字长为5位（含1位符号位， $n=4$ ）， $x=0.1011$ ， $y=0.1101$ ，采用原码恢复余数法求 x/y

$|x|=0.1011$ ， $|y|=0.1101$ ， $[|y|]_{\text{补}}=0.1101$ ， $[-|y|]_{\text{补}}=1.0011$

符号单独处理：符号位 = $x_s \oplus y_s$

实现方法：上商0/1，得到余数，余数末尾补0

数值位取绝对值进行除法计算

```
      0.1101
01101 √ 01011
      00000
      10110
      01101
      10010
      01101
      01010
      00000
      10100
      01101
      0111
```

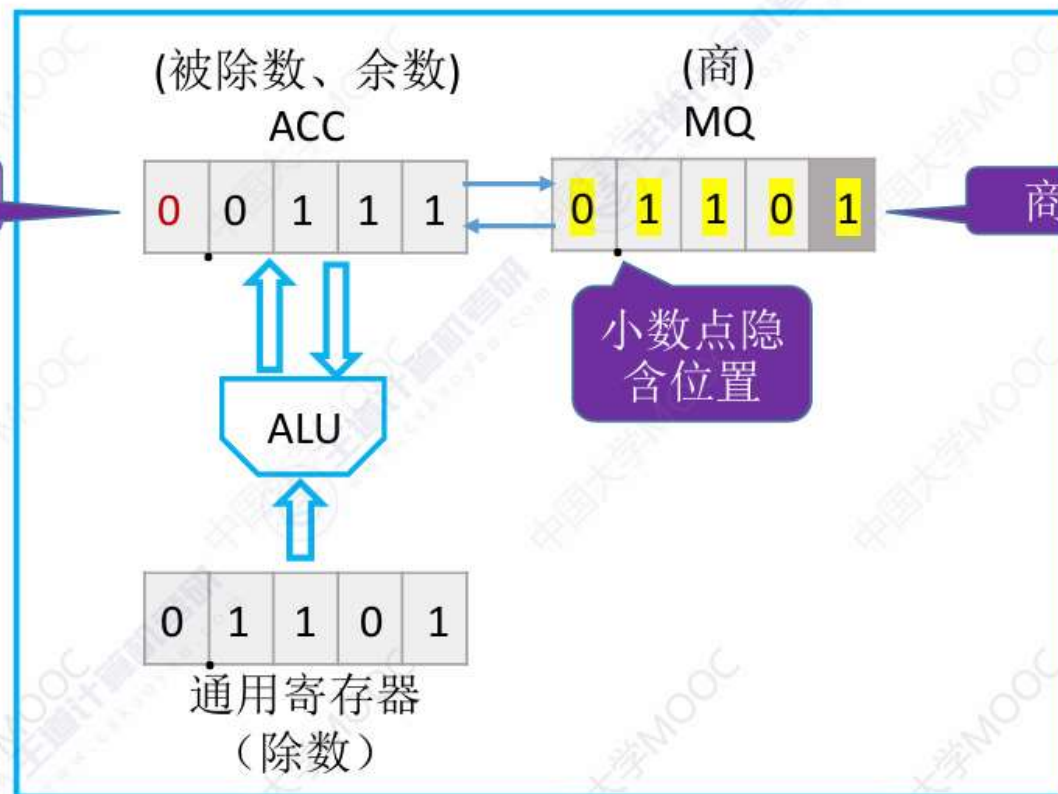
余数 = 0.0111×2^{-n}

商 = 0.1101

小数点隐含位置

运算器

计算机很傻，会先默认上商1，如果搞错了再改上商0。并“恢复余数”



原码除法：加减交替法

又名：不恢复余数法

符号位与数值位分开处理

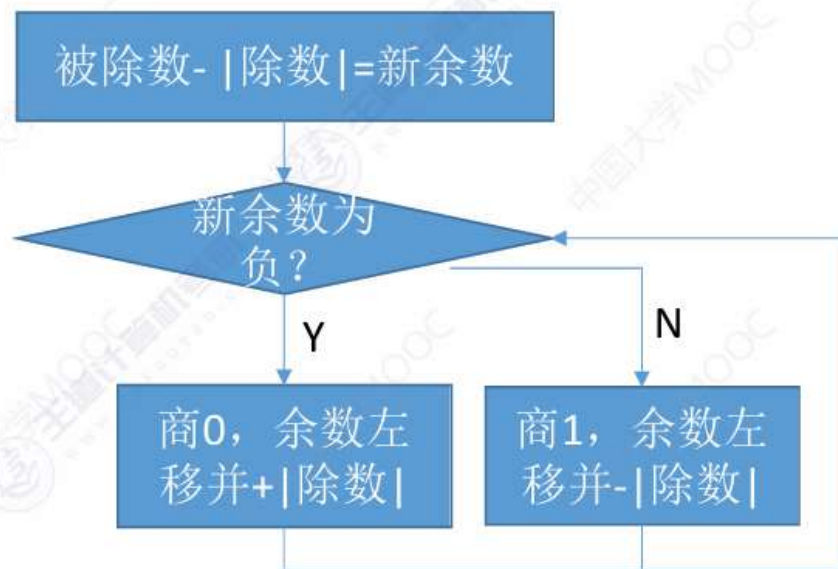
设机器字长为5位（含1位符号位， $n=4$ ）， $x=0.1011$ ， $y=0.1101$ ，采用原码加减交替除法求 x/y

$|x|=0.1011$ ， $|y|=0.1101$ ， $[|y|]_{\text{补}}=0.1101$ ， $[-|y|]_{\text{补}}=1.0011$

$Q_s = x_s \oplus y_s = 0 \oplus 0 = 0$
得 $x/y = +0.1101$
余 0.0111×2^{-4}

被除数/余数	商
0.1011	
$+[- y]_{\text{补}}$	
1.0011	
1.1110	0
左移	
1.1100	
$+ [y]_{\text{补}}$	
0.1101	
0.1001	01
左移	
1.0010	
$+ [- y]_{\text{补}}$	
1.0011	
0.0101	011
左移	
0.1010	
$+ [- y]_{\text{补}}$	
1.0011	
1.1101	0110
左移	
1.1010	
$+ [y]_{\text{补}}$	
0.1101	
0.0111	01101

若余数为负，
需商0，并
 $+ [|y|]_{\text{补}}$ 得到
正确余数



加/减 $n+1$ 次，每次加减确定一位商；
左移 n 次（最后一次加减完不移位）
最终可能还要再多一次加



除 法 类 型	符号位参与 运算	加 减 次 数	移 位		上商、加减原则	说 明
			方 向	次 数		
原码加减交 替法	否	$N+1$ 或 $N+2$	左	N	余数的正负	若最终余数为负， 需恢复余数
补码加减交 替法	是	$N+1$	左	N	余数和除数是否 同号	商末位恒置1

2.3.1 无符号整数的表示与零扩展

表示形式：所有二进制位均为数值位，没有符号位。n 位无符号整数表数范围为： $0 \sim 2^n - 1$ (如 8位: $0 \sim 255$)。

❏ 零扩展 (Zero Extension)：

当小字长无符号数转换为大字长时 (例如 C语言中 unsigned short 转 unsigned int)，高位一律填充 0。

⚠ 溢出条件：无符号数加法最高位产生进位 ($C_{out} = 1$) 即为溢出 ($CF = 1$)。

2.3.2 带符号整数与符号扩展

± 表示形式：最高位为符号位，计算机内部统一用补码表示与存储。n 位带符号整数范围： $-2^{n-1} \sim 2^{n-1}-1$ 。

符号扩展 (Sign Extension)：

当小字长带符号数转换为大字长时 (例如 short 转 int)，高位填充原符号位的值：

- 正数 (符号位0)：高位全部补 0；
- 负数 (符号位1)：高位全部补 1。

2.3.2 补码溢出判断的三种判定方法



单符号位观察法

仅同号相加或异号相减时可能溢出。

- 正 + 正 = 负 → 上溢
- 负 + 负 = 正 → 下溢



双符号位法 (变形补码)

符号位用 2 位表示 (00正, 11负):

- 01 表示正溢出 (上溢)
- 10 表示负溢出 (下溢)



进位异或判别法

观察最高数值位进位 C_1 与符号位进位 C_0 :

$$OF = C_1 \oplus C_0$$

若为 1 则表示产生溢出。

2.3.2 C 语言类型转换与截断/扩展陷阱

转换场景	硬件位级转换规则	例题与结果 (32位系统)
signed ↔ unsigned (同字长)	位模式不变，仅改变对最高符号位的解释方式。	int x = -1; (0xFFFFFFFF) 转为 unsigned int → 4294967295
大字长 → 小字长 (截断)	直接高位截断 (保留低位)。	int x = 32768; (0x00008000) 转为 short → -32768 (0x8000)
小字长 → 大字长 (扩展)	无符号零扩展；带符号符号扩展。	short x = -1; (0xFFFF) 转为 int → -1 (0xFFFFFFFF)

大小端模式

多字节数据在内存里一定是占连续的几个字节

最高有效字节 (MSB)

最低有效字节 (LSB)

4字节 int: 01 23 45 67 H

19088743 D

0000 0001 0010 0011 0100 0101 0110 0111 B

便于人类阅读

大端方式

	0800H	0801H	0802H	0803H	
...	01H	23H	45H	67H	...

便于机器处理

小端方式

	0800H	0801H	0802H	0803H	
...	67H	45H	23H	01H	...

边界对齐

现代计算机通常是按字节编址，即每个字节对应1个地址
通常也支持按字、按半字、按字节寻址。

假设存储字长为32位，则1个字=32bit，半字=16bit。每次访存只能读/写1个字

字节 1	字节 2	字节 3	填充
半字 1		半字 2	
半字 3		填充	
字 1			

访问一个字/半字
都只需一次访存

图 2.10 边界对齐方式

字节 1	字节 2	字节 3	半字 1-1
半字 1-2	半字 2		半字 3-1
半字 3-2	字 1-1		
字 1-2			

访问一个字/半字
可能要两次访存

图 2.11 边界不对齐方式

2.4.1 浮点数的基本表示格式与数域

浮点数类似于科学计数法，其通用数学表达式为：

$$N = (-1)^S \times M \times R^E$$

- S (Sign): 数符, 0 表示正, 1 表示负。
- E (Exponent): 阶码, 决定浮点数的表示范围。
- M (Significand/Mantissa): 尾数, 决定浮点数的精度。

范围与精度的折衷 (Trade-off)

- 字长固定时:
 - 阶码位数越多 → 表数范围越大, 但精度降低。
 - 尾数位数越多 → 表示精度越高, 但范围减小。

2.4.1 IEEE 754 标准参数对比表格

类型	总位数	数符 (S)	阶码 (E) / 偏置值 (Bias)	尾数 (M) / 隐藏位
单精度 (float)	32 bit	1 bit	8 bit / 127 (0x7F)	23 bit / 隐藏最高位 1.M
双精度 (double)	64 bit	1 bit	11 bit / 1023 (0x3FF)	52 bit / 隐藏最高位 1.M

IEEE 754 单精度浮点数真值计算公式:

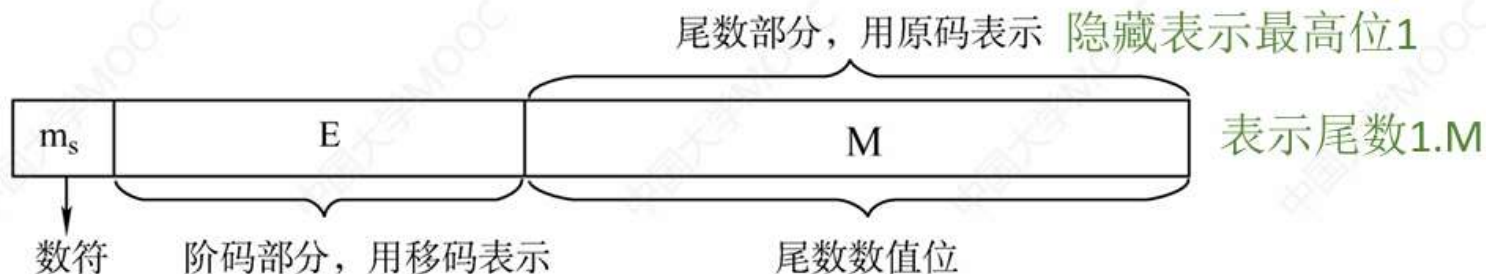
$$V = (-1)^S \times (1 + M) \times 2^{E - 127}$$

2.4.1 IEEE 754 特殊值与非规格化数

阶码 E (8位)	尾数 M (23位)	所表示的数值 / 含义
E = 255 (全1)	M ≠ 0	NaN (Not a Number) 非法数值操作结果
E = 255 (全1)	M = 0	±∞ (无穷大) 阶码上溢产生
0 < E < 255	任意 M	规格化数: $V = (-1)^S \times (1.M) \times 2^{E-127}$
E = 0 (全0)	M ≠ 0	非规格化数: $V = (-1)^S \times (0.M) \times 2^{-126}$ (用于渐进下溢)
E = 0 (全0)	M = 0	±0 (正负零)

IEEE 754标准

天然地完成了“规格化”



IEEE 754 单精度浮点型能表示的最小绝对值、最大绝对值是多少？

若要表示的数绝对值还要更小，怎么办？

最小绝对值：尾数全为0，阶码真值最小-126，对应移码机器数 0000 0001
此时整体的真值为 $(1.0)_2 \times 2^{-126}$

最大绝对值：尾数全为1，阶码真值最大 127，对应移码机器数 1111 1110
此时整体的真值为 $(1.111...11)_2 \times 2^{127}$

格 式	规格化的最小绝对值	规格化的最大绝对值
单精度	$E=1, M=0: 1.0 \times 2^{1-127}=2^{-126}$	$E=254, M=.11...1: 1.11...1 \times 2^{254-127}=2^{127} \times (2-2^{-23})$
双精度	$E=1, M=0: 1.0 \times 2^{1-1023}=2^{-1022}$	$E=2046, M=.11...1: 1.11...1 \times 2^{2046-1023}=2^{1023} \times (2-2^{-52})$

2.4.2 浮点数加/减运算五步法

1

1. 对阶

小阶向大阶看齐
尾数右移

2

2. 尾数求和

带符号位做
补码加/减

3

3. 规格化

左规或右规
调整最高有效位

4

4. 舍入 & 溢出判断

0舍1入 / 恒置1
阶码上溢/下溢检查

2.4.2 浮点加减：对阶与尾数运算细节

第 1 步：对阶 (Alignment)

- 计算阶差 $\Delta E = E_x - E_y$ 。
- 原则：小阶向大阶看齐！小阶码的尾数右移 ΔE 位，阶码增加 ΔE 。
- 为什么不能大阶向小阶看齐？大阶向小阶看齐会导致高位有效数字丢失！

第 2 步：尾数求和 (Addition)

- 对阶完成后，隐藏的 '1' (1.M) 显式恢复。
- 将尾数转化为补码形式，进行定点补码加减运算，得出初步尾数和。

2.4.2 浮点加减：左规与右规逻辑

→右规 (Right Normalization)

- 触发条件：尾数相加产生溢出 (双符号位为 01 或 10)。
- 操作：尾数右移 1 位，阶码 +1。
- 注意：右规最多只需进行 1 次！

←左规 (Left Normalization)

- 触发条件：尾数相减导致最高有效位为 0 (如 00.001... 或 11.110...)。
- 操作：尾数左移 1 位，阶码 -1，直到最高位为 1。
- 注意：左规可能需要进行多次！

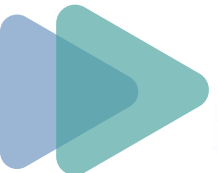
2.4.2 舍入模式与阶码溢出判定

常见舍入处理方法

- ✓ **0 舍 1 入法**：类似四舍五入。若移出的最高位为 1，尾数末位加 1 (可能再次引发右规)。
- ✓ **恒置 1 法**：只要有被移出的位数，尾数末位硬性置 1。

阶码溢出判定

- ⚠ **阶码上溢 (Overflow)**：阶码超过最大值 ($E > 254$)，触发中断 / 变为 $\pm\infty$ 。
- ⚠ **阶码下溢 (Underflow)**：阶码小于最小值 ($E < 0$)，按 0 或非规格化数处理。



23. 【2019 统考真题】下列关于冯·诺依曼计算机基本思想的叙述中，错误的是（ ）。

- A. 程序的功能都通过中央处理器执行指令实现
- B. 指令和数据都用二进制数表示，形式上无差别
- C. 指令按地址访问，数据都在指令中直接给出
- D. 程序执行前，指令和数据需预先存放在存储器中

23. C

冯·诺依曼结构计算机的功能部件包括输入设备、输出设备、存储器、运算器和控制器，程序的功能都通过中央处理器（运算器和控制器）执行指令，A 正确。指令和数据以同等地位存放于存储器内，形式上无差别，只在程序执行时具有不同的含义，B 正确。指令按地址访问，数据由指令的地址码指出，除立即寻址外，数据均存放在存储器内，C 错误。在程序执行前，指令和数据需预先存放在存储器中，中央处理器可以从存储器存取代码，D 正确。



24. 【2022 统考真题】将高级语言源程序转换为可执行目标文件的主要过程是 ()。

A. 预处理→编译→汇编→链接

B. 预处理→汇编→编译→链接

C. 预处理→编译→链接→汇编

D. 预处理→汇编→链接→编译

24. A

将源程序转换为可执行目标文件的过程分为预处理、编译、汇编、链接四个阶段。

- 
31. 【2023 统考真题】若机器 M 的主频为 1.5GHz，在 M 上执行程序 P 的指令条数为 5×10^5 ，P 的平均 CPI 为 1.2，则 P 在 M 上的指令执行速度和用户 CPU 时间分别为（ ）。
A. 0.8GIPS, 0.4ms B. 0.8GIPS, 0.4 μ s C. 1.25GIPS, 0.4ms D. 1.25GIPS, 0.4 μ s

31. C

程序 P 的指令条数为 5×10^5 ，平均 CPI 为 1.2，程序 P 的总时钟周期数为 $5 \times 10^5 \times 1.2 = 6 \times 10^5$ ，主频 1.5GHz 说明 1s 有 $1.5\text{G} = 1.5 \times 10^9$ 个时钟周期。因此，指令执行速度 = 主频/平均 CPI = $1.5\text{G} \div 1.2 = 1.25\text{GIPS}$ ，用户 CPU 时间 = $6 \times 10^5 \div (1.5 \times 10^9)\text{s} = 4 \times 10^{-4}\text{s} = 0.4\text{ms}$ 。

32. 【2019 统考真题】考虑以下 C 语言代码:

```
unsigned short usi = 65535;  
short si = usi;
```

执行上述程序段后, si 的值是 ()。

- A. -1 B. -32767 C. -32768 D. -65535

32. A

unsigned short 型为无符号短整型, 长度为 2 字节, 因此 unsigned short usi 型转换为二进制代码即 1111 1111 1111 1111。short 型为短整型, 长度为 2 字节, 在采用补码的机器上, short si 的二进制代码为 1111 1111 1111 1111, 因此 si 的值为-1。

33. 【2021 统考真题】已知有符号整数用补码表示，变量 x, y, z 的机器数分别为 FFFDH, FFDFH, 7FFCH，下列结论中，正确的是（ ）。

- A. 若 x, y 和 z 为无符号整数，则 $z < x < y$
- B. 若 x, y 和 z 为无符号整数，则 $x < y < z$
- C. 若 x, y 和 z 为有符号整数，则 $x < y < z$
- D. 若 x, y 和 z 为有符号整数，则 $y < x < z$

33. D

若 x, y 和 z 均为无符号整数，则 $x > y > z$ ，A 和 B 错误。若 x, y 和 z 均为有符号整数，补码的最高位是符号位，0 表示正数，1 表示负数，因此 z 为正数，而 x 和 y 为负数。对于 x 和 y 的比较，数值位取反加 1，可知 $x = -3$ ， $y = -33$ ，故 $x > y$ 。D 正确。

边界对齐

一句话：K字节大小的数据必须要存储在K的整数倍的地址上

例如一个存储字长为32位的机器，现来详述一下依次存入一个int,short,double,char,short类型的数据的过程。假定int,short,double,char分别为4, 2, 8, 1字节。那么它们分别占据字、半字、双字、字节。

字地址：4的倍数

半字地址：2的倍数

双字地址：8的倍数

字节地址：任意

那么第一个int就正好占据一个字，第一行就全是int.

short是半字，地址要是2的倍数，随意要占据第二行的前两格。

double是双字，地址要是8的倍数，第三行第一个正好是8，所以占据3, 4两行。

char是字节，任意，所以占据double后面的一个格子就行了，这里正好是第五行的第一格。

short是半字，地址要是2的倍数，而char后面的那个地址是17，所以short的首地址不可以正好在char后面，要空一格，从18开始的两个格子。

边界对齐后插入数的位置如图所示：

地址	0字节	1字节	2字节	3字节
0	int	int	int	int
4	short	short		
8	double	double	double	double
12	double	double	double	double
16	char		short	short

边界对齐是以空间换时间的。

34. 【2022 统考真题】32 位补码所能表示的整数范围是 ()。

- A. $-2^{32} \sim 2^{31} - 1$ B. $-2^{31} \sim 2^{31} - 1$ C. $-2^{32} \sim 2^{32} - 1$ D. $-2^{31} \sim 2^{32} - 1$

34. B

n 位补码整数的最小值是 $1,00\dots 0$ (即 -2^{n-1})；最大值是 $0,11\dots 1$ (即 $2^{n-1} - 1$)。 n 位补码整数所能表示的范围是 $-2^{n-1} \sim 2^{n-1} - 1$ ，32 位补码整数所能表示的范围是 $-2^{31} \sim 2^{31} - 1$ 。

29. 【2018 统考真题】假定有符号整数采用补码表示，若 int 型变量 x 和 y 的机器数分别是 FFFF FFDFH 和 0000 0041H，则 x、y 的值及 x-y 的机器数分别是 ()。

- A. $x = -65, y = 41$, x-y 的机器数溢出
- B. $x = -33, y = 65$, x-y 的机器数为 FFFF FF9DH
- C. $x = -33, y = 65$, x-y 的机器数为 FFFF FF9EH
- D. $x = -65, y = 41$, x-y 的机器数为 FFFF FF96H

29. C

利用补码转换成原码的规则：负数的符号位不变数值位取反加 1；正数补码等于原码。两个机器数对应的原码是 $[x]_{\text{原}} = 80000021\text{H}$ ，对应的数值是 -33， $[y]_{\text{原}} = [y]_{\text{补}} = 00000041\text{H} = 65$ 。排除 A、D。x-y 直接利用补码减法准则， $[x]_{\text{补}} - [y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}}$ ，-y 的补码是连同符号位取反加 1，最终减法变成加法，得出结果为 FFFFFFF9EH。



30. 【2018 统考真题】整数 x 的机器数为 1101 1000，分别对 x 进行逻辑右移 1 位和算术右移 1 位操作，得到的机器数各是（ ）。

A. 1110 1100、1110 1100

B. 0110 1100、1110 1100

C. 1110 1100、0110 1100

D. 0110 1100、0110 1100

30. B

逻辑移位：左移和右移空位都补 0，且所有数字参与移动；补码算术移位：仍然是所有数字参与移动，右移空位补符号位，左移空位补 0。根据该规则，轻松选出 B。

31. 【2018 统考真题】减法指令“sub R1, R2, R3”的功能为“(R1) - (R2) → R3”，该指令执行后将生成进位/借位标志 CF 和溢出标志 OF。若(R1) = FFFF FFFFH, (R2) = FFFF FFF0H, 则该减法指令执行后, CF 与 OF 分别为 ()。

A. CF = 0, OF = 0 B. CF = 1, OF = 0 C. CF = 0, OF = 1 D. CF = 1, OF = 1

31. A

$[x]_{\text{补}} - [y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}}$, $[-R2]_{\text{补}} = 00000010\text{H}$, 很明显 $[R1]_{\text{补}} + [-R2]_{\text{补}}$ 的最高位进位和符号位进位都是 1 (当最高位进位和符号位进位的值不相同才产生溢出), 可以判断溢出标志 OF 为 0。同时, 减法操作只需判断借位标志, R1 大于 R2, 所以借位标志为 0。

48. 【2022 统考真题】 -0.4375 的 IEEE 754 单精度浮点数表示为 ()。

- A. BEE0 0000H B. BF60 0000H C. BF70 0000H D. C0E0 0000H

48. A

IEEE 754 单精度浮点数格式中依次为符号 1 位、阶码 8 位（偏置值 127）、尾数 23 位（隐藏 1 位）。 $-0.4375 = -1.75 \times 2^{-2}$ ，保证小数点前是 1。根据单精度浮点数格式，符号为 1；阶码为移码表示， $-2 + 127 = 125$ ，写成 8 位二进制数为 01111101；尾数隐藏小数点前的 1，剩下的 0.75 写成二进制数为 0.11，所以尾数部分是 1100...0。该浮点数的二进制格式为 1011 1110 1110 0000 0000 0000 000 0000，对应的十六进制格式为 BEE0 0000H。

50. 【2023 统考真题】已知 float 型变量用 IEEE 754 单精度浮点数格式表示。若 float 型变量 x 的机器数为 8020 0000H, 则 x 的值是 ()。

A. -2^{-128}

B. -1.01×2^{-127}

C. -1.01×2^{-126}

D. 非数 (NaN)

50. A

把 x 的机器数按二进制展开, $8020\ 0000H = \underline{1000\ 0000\ 0010\ 0000\ 0000\ 0000\ 0000\ 0000}B$, 符号为负, 阶码全为 0, 尾数不全为 0, 由下表可知, 这是非规格化数, 对于 32 位非规格化负数, 若尾数的二进制为 f , 则真值为 $-2^{-126} \times 0.f = -2^{-126} \times 0.01 = -2^{-128}$ 。

值的类型	单精度 (32 位)			
	符号	阶码	尾数	值
非规格化正数	0	0	$f \neq 0$	$2^{-126}(0.f)$
非规格化负数	1	0	$f \neq 0$	$-2^{-126}(0.f)$